PRASAD V POTLURI SIDDHARTHA INSTITUTE OF TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

ACADEMIC YEAR: 2026-27

DEEP LEARNING COURSE PROJECT REPORT

FOOD FRESHNESS DETECTION USING DEEP LEARNING (MOBILENETV2 TRANSFER LEARNING)

Faculty Name: Ms. G. Lalitha Kumari

Team Members:

1. **Name: K. Jan Vikash** — **Roll No: 23501A0571**
2. **Name: K. Sruthi** — **Roll No: 23501A0573**
3. **Name: K. Yoshita Navya** — **Roll No: 23501A0581**
4. **Name: K. Harsha Vardhan** — **Roll No: 23501A0584**

ABSTRACT

Food waste and food safety are serious concerns in households, markets and supply chains. Fruit that has begun to rot can spread decay to other produce and can be unsafe to eat, yet freshness is usually judged by eye, which is slow and subjective. Traditional inspection methods may also be inconsistent because they depend on the person, the lighting and the time available.

This project presents a food freshness detection system that classifies fruit images as Fresh or Rotten using deep learning. The system uses MobileNetV2, a convolutional neural network pre-trained on the ImageNet dataset, adapted through transfer learning. The model is trained on the publicly available Fresh and Rotten Fruits dataset (Sultana, Jahan and Uddin, 2022), which is obtained automatically from the Hugging Face Hub. The original (raw) subset of 3,200 photographs, 1,600 fresh and 1,600 rotten, covering eight types of fruit, is used. Each image is resized to 224 × 224 pixels and scaled to the range expected by MobileNetV2. Data augmentation is applied to training images only. The dataset is divided into training, validation and testing sets using stratified splitting before any augmentation to avoid data leakage.

The model contains the pre-trained MobileNetV2 base, a global average pooling layer, a dropout layer and a single sigmoid output neuron that gives the probability that the fruit is rotten. Training is carried out in two phases: first only the new classifier head is trained while the base is frozen, and then the top 30 layers of the base are fine-tuned with a very small learning rate. Early stopping and model checkpointing are used. Model performance is evaluated using accuracy, precision, recall, F1-score, ROC-AUC and a confusion matrix. The trained model is saved and used by a Streamlit web application, which allows a user to upload or capture a fruit photograph and view the predicted class and confidence score. A Gradio interface is also provided inside the Google Colab notebook.

Unlike traditional methods that depend on manually designed colour or texture features, a convolutional neural network learns visual patterns directly from the images. These patterns may include discolouration, dark spots, wrinkled skin and changes in texture that are commonly associated with spoiled fruit. Using a pre-trained network allows good results to be obtained from a relatively small dataset and a short training time. However, the effectiveness of the model depends on the quality, diversity and representativeness of the training images.

KEYWORDS: *Food Freshness Detection, Deep Learning, Convolutional Neural Network (CNN), Transfer Learning, MobileNetV2, Image Classification, Fresh and Rotten Fruits, Streamlit.*

INTRODUCTION

Fruits are an important part of a healthy diet and an important agricultural product. A large share of fresh produce is lost between the farm and the consumer because it spoils before it is eaten or sold. Consumers, shop owners and food processing industries therefore need to identify rotten fruit quickly, so that it can be removed before it contaminates other items or is eaten.

In practice, freshness is usually judged by looking at the fruit. A person checks the colour, the skin and the presence of spots or mould. This approach is simple, but it is subjective, it can differ from one person to another, and it is difficult to apply to large quantities of fruit.

Traditional automated techniques use manually designed image features such as colour histograms and texture descriptors together with classifiers such as Support Vector Machines. Their performance depends on the chosen features and they may not work well when lighting, background or camera conditions change. More advanced methods such as hyperspectral imaging or electronic noses can be accurate, but the equipment is expensive and not easily available to ordinary users.

Deep learning provides another approach to freshness detection by learning patterns directly from data. In this project, a convolutional neural network based on MobileNetV2 is used. Instead of depending on manually extracted features, the network learns visual characteristics that help distinguish fresh fruit from rotten fruit. Transfer learning is used so that the network can reuse general visual knowledge learned from the large ImageNet dataset.

The system first obtains the fruit image dataset, preprocesses the images, trains the model in two phases and evaluates its performance on unseen test images. Finally, it accepts a fruit photograph from the user through a web interface and produces the predicted class and a confidence score.

Furthermore, the proposed system aims to make freshness checking more accessible by allowing users to check a fruit using only a photograph, without any technical knowledge. The performance of the system is assessed using multiple evaluation metrics to understand its reliability and limitations. This approach can serve as a foundation for more advanced applications in food quality monitoring, smart refrigerators and automated sorting systems.

PROBLEM STATEMENT

Manual inspection of fruit is slow, subjective and inconsistent. Rotten fruit that is not identified early increases food waste and may be a health risk. The appearance of spoilage also varies between fruit types and depends on lighting and camera conditions, which makes it difficult to define simple rules for detecting it.

Therefore, there is a need for an automated system that can learn visual patterns from fruit images and classify a fruit as fresh or rotten. The proposed project addresses this problem by developing a MobileNetV2-based convolutional neural network, trained using transfer learning, that predicts whether a given fruit image shows fresh or rotten fruit and provides a confidence score for the prediction.

Objectives of the Project

The main objectives of this project are:

1. To develop an automated system for classifying fruit images as fresh or rotten.
2. To obtain a publicly available fresh and rotten fruit dataset and load it automatically in Google Colab.
3. To preprocess the images by resizing them to 224 × 224 pixels and scaling pixel values correctly for MobileNetV2.
4. To apply data augmentation to the training images only, in order to improve generalization.
5. To build a classification model using transfer learning with the pre-trained MobileNetV2 network.
6. To divide the data into training, validation and test sets without data leakage, and to train the model using early stopping and model checkpointing.
7. To evaluate the trained model using accuracy, precision, recall, F1-score, ROC-AUC and a confusion matrix.
8. To provide a web interface that accepts an uploaded image and shows the predicted class and confidence score.
9. To save the trained model so that it can be reused without retraining, and to deploy the application using Streamlit and GitHub.

LITERATURE REVIEW

Fruit quality and freshness classification has been studied using manual inspection, traditional image processing with machine learning, specialized sensing techniques and deep learning approaches.

Manual inspection: Trained people judge fruit by its appearance. This is simple and requires no equipment, but it is subjective, tiring and difficult to scale.

Feature-based machine learning: These approaches extract features such as colour histograms, texture and shape from images, and then use a classifier such as an SVM or k-NN to predict the class. Their performance depends on the selected features and may decrease when imaging conditions change.

Specialized sensing: Techniques such as hyperspectral imaging, near-infrared spectroscopy and electronic noses can detect quality changes that are not visible to the eye. However, the equipment is expensive and is not convenient for everyday use.

Deep learning-based detection: Convolutional neural networks learn features directly from images and have been widely applied to image classification, including fruit classification. Deep networks may be trained from scratch or adapted through transfer learning from large pre-trained models.

Transfer learning with lightweight networks: Networks such as MobileNetV2, which was designed to be efficient, can be pre-trained on ImageNet and then adapted to a new task. This approach often gives good accuracy even when the new dataset is relatively small.

Based on these approaches, this project implements a transfer-learning model using MobileNetV2 for binary classification of fruit images. The dataset used was published by Sultana, Jahan and Uddin (2022) specifically to support machine-based evaluation of fruit quality. A formal comparative study against other architectures such as VGG16, ResNet or a CNN trained from scratch would require implementing and evaluating those models separately.

EXISTING SYSTEM

Existing approaches for judging the freshness of fruit include manual inspection, traditional image processing combined with machine learning classifiers, and specialized sensor-based equipment.

Manual inspection is performed by farmers, vendors and consumers, and depends on personal experience. Classical machine learning systems require features such as colour and texture to be designed and extracted before classification. Sensor-based systems, such as hyperspectral cameras and electronic noses, give detailed information but require special hardware.

These approaches are useful, but their effectiveness depends on the person or the selected features, the available equipment and the imaging conditions. This project explores a deep learning approach that learns visual patterns directly from ordinary photographs.

LIMITATIONS OF THE EXISTING SYSTEM

The following are common limitations of existing approaches:

1. **Subjectivity:** Manual inspection varies between people and depends on experience and fatigue.
2. **Manual feature design:** Classical machine learning systems require suitable image features to be identified and extracted.
3. **Sensitivity to conditions:** Changes in lighting, background and camera angle can reduce the accuracy of simple feature-based methods.
4. **Cost of equipment:** Hyperspectral imaging and electronic noses are expensive and not accessible to most users.
5. **Limited scalability:** Manual inspection is slow when large quantities of fruit must be checked.
6. **False predictions:** A fresh fruit may be wrongly rejected, or a rotten fruit may be missed.

These limitations motivate the exploration of deep learning techniques that learn patterns from images. However, the proposed system can also make errors and must be evaluated using appropriate test data.

PROPOSED SYSTEM

The proposed system uses a MobileNetV2-based convolutional neural network, trained with transfer learning, to classify fruit images as fresh or rotten. The fresh and rotten fruit dataset is downloaded automatically from the Hugging Face Hub, the original (raw) images are exported into fresh and rotten folders, and the binary labels are created from the folder names.

Each image is resized to 224 × 224 pixels. The dataset is divided into training, validation and testing sets while preserving the class and fruit proportions. Data augmentation, including random flipping, rotation, zoom and contrast change, is applied to the training images only.

The model is constructed from a pre-trained MobileNetV2 base, a global average pooling layer, a dropout layer and a sigmoid output layer. It is trained using binary cross-entropy loss and the Adam optimizer in two phases: head training with a frozen base, followed by fine-tuning of the top layers. Early stopping and model checkpointing help manage training.

After training, the model is evaluated using a separate test set. The system calculates classification metrics and generates visualizations such as the confusion matrix, ROC curve and training curves. The trained model is then saved and used by a Streamlit web application, which classifies a fruit image entered by the user and displays the predicted class and confidence.

ADVANTAGES OF THE PROPOSED SYSTEM

1. Learns visual patterns directly from images instead of relying on manually designed features.
2. Uses transfer learning, so good performance can be obtained from a relatively small dataset.
3. MobileNetV2 is lightweight, so training and prediction are fast.
4. Supports binary classification of fresh and rotten fruit and produces a confidence score.
5. Uses separate training, validation and test sets, with augmentation applied to training data only.
6. Provides several performance measures that help assess classification quality.
7. Saves the trained model so it can be reused without retraining.
8. Provides a simple web interface that can be deployed online using Streamlit and GitHub.

PROPOSED SYSTEM ARCHITECTURE

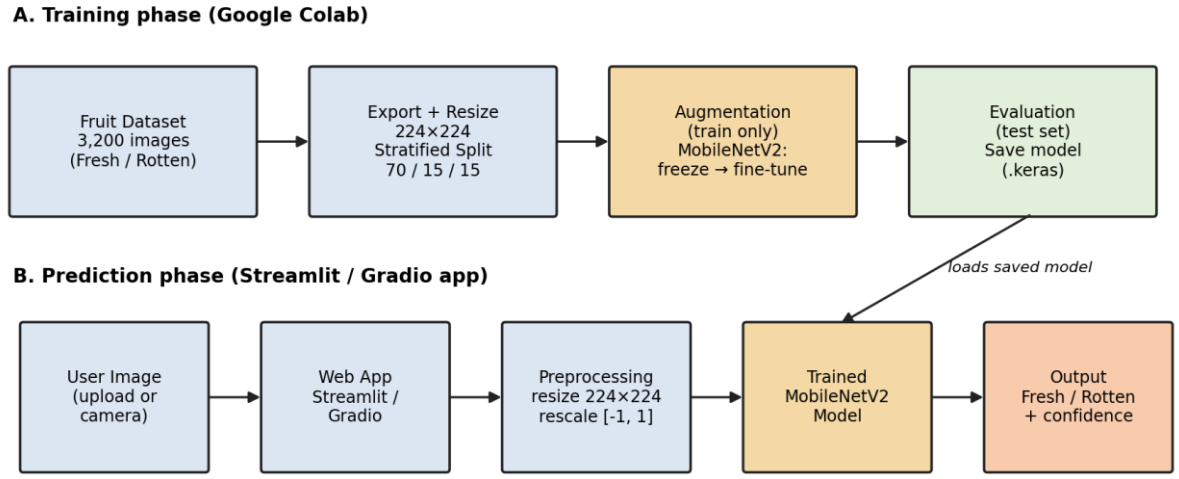


Figure 1: Proposed system architecture

The architecture has two phases. In the training phase, the dataset is downloaded, preprocessed, split, augmented and used to train and evaluate the MobileNetV2 model, which is then saved. In the prediction phase, the user supplies a fruit image through the web application, the image is preprocessed in the same way as the training data, and the saved model returns the probability that the fruit is rotten.

SYSTEM REQUIREMENTS

Hardware requirements

- Computer or laptop
- Processor capable of running Python and TensorFlow
- Sufficient RAM for image processing and model training (a GPU, such as the free Google Colab T4 GPU, speeds up training)
- Internet connection for downloading the dataset and installing dependencies

Software requirements

- Python
- Google Colab or a compatible Jupyter Notebook environment
- TensorFlow and Keras
- NumPy and Pandas
- Matplotlib
- Scikit-learn
- Pillow (PIL)
- Hugging Face datasets library
- Gradio (interface inside the notebook)
- Streamlit (web application)

- Git and GitHub (version control and deployment)

Programming language: Python

Deep learning framework: TensorFlow/Keras

Dataset source: Fresh and Rotten Fruits Dataset (Sultana, Jahan and Uddin, 2022), through the Hugging Face Hub

DATASET DESCRIPTION

This project uses the Fresh and Rotten Fruits Dataset for Machine-Based Evaluation of Fruit Quality, published by Nusrat Sultana, Musfika Jahan and Mohammad Shorif Uddin (Data in Brief, 2022; Mendeley Data DOI 10.17632/bdd69gyhv8.1). The photographs were collected from fruit shops and real fields, and the fresh and rotten labels were assigned with the help of a domain expert from an agricultural institute. A reformatted copy of the dataset, released under the CC BY 4.0 licence, is available on the Hugging Face Hub as Project-AgML/fresh_rotten_fruit_classification. The notebook loads it using the load_dataset function of the Hugging Face datasets library.

The Hugging Face version provides two subsets: a raw subset containing the original photographs, and an augmented subset containing additional artificially modified copies. This project uses only the raw subset, because augmented copies of the same photograph could appear in both the training and test sets and make the results look better than they really are. The raw subset contains:

- Total images: 3,200
- Fresh images: 1,600 and rotten images: 1,600
- Fruit types: 8 (apple, banana, orange, grape, guava, jujube, pomegranate and strawberry)
- Each record contains the image, a fresh/rotten label and the fruit type

The notebook exports the images into the following folder structure, resizing each image to 224 × 224 pixels and saving it as a JPEG file:

fruit_data/fresh/ (for example apple_00000.jpg, banana_00012.jpg, ...)

fruit_data/rotten/ (for example apple_01600.jpg, orange_01753.jpg, ...)

The target labels are mapped to two classes:

- Class 0: Fresh fruit
- Class 1: Rotten fruit

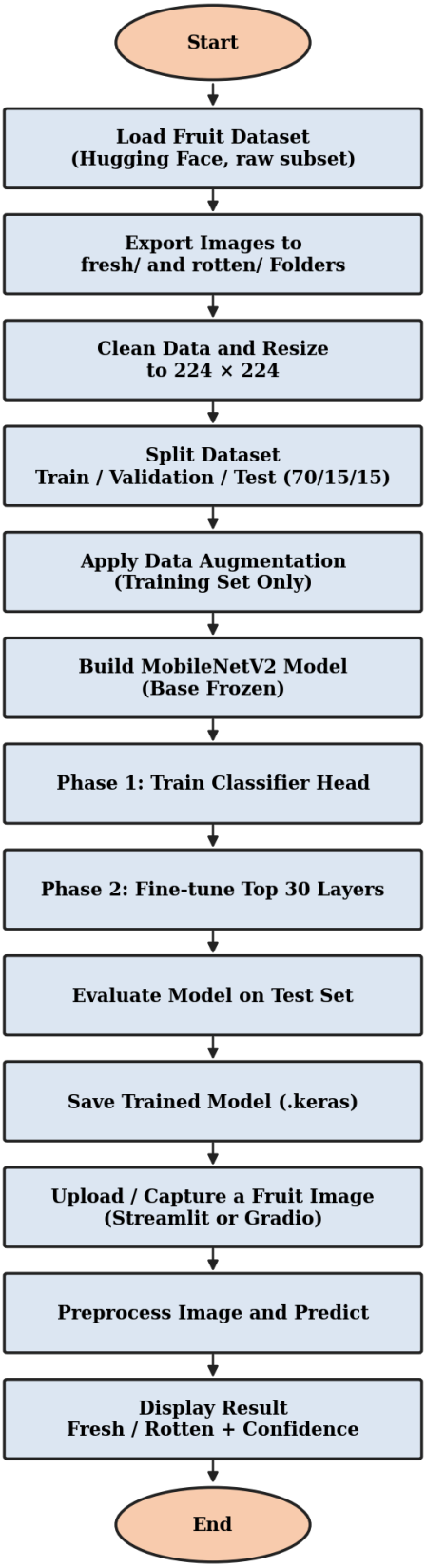
The fruit type is kept in each file name and is used only to make the training, validation and test sets balanced. The notebook can optionally be restricted to apples, bananas and oranges by changing one setting. The actual number of images in each subset is printed when the notebook is executed.

METHODOLOGY

The project follows a sequence of steps, from dataset collection to fruit image prediction.

1. Download the fresh and rotten fruit dataset
2. Export images and create binary labels
3. Clean and explore the dataset
4. Resize images to 224 × 224
5. Split into training, validation and test sets
6. Apply data augmentation to training images only
7. Build the MobileNetV2 transfer-learning model
8. Train the model in two phases (head training and fine-tuning)
9. Evaluate the model performance
10. Predict the class of a new fruit image

FLOW CHART:


Start
Load Fruit Dataset
(Hugging Face, raw subset)
Export Images to
fresh/ and rotten/ Folders
Clean Data and Resize
to 224 × 224
Split Dataset
Train / Validation / Test (70/15/15)
Apply Data Augmentation
(Training Set Only)
Build MobileNetV2 Model
(Base Frozen)
Phase 1: Train Classifier Head
Phase 2: Fine-tune Top 30 Layers
Evaluate Model on Test Set
Save Trained Model (.keras)
Upload / Capture a Fruit Image
(Streamlit or Gradio)
Preprocess Image and Predict
Display Result
Fresh / Rotten + Confidence
End

DATA PREPROCESSING

Data preprocessing converts the raw fruit images into a consistent format that can be used by the MobileNetV2 model.

First, the notebook exports the images from the dataset, converts each image to the three-channel RGB format and resizes it to 224 × 224 pixels, which is the input size used by the pre-trained MobileNetV2 network. Files that cannot be read are skipped and counted.

Next, the labels are prepared. The folder name of each image (fresh or rotten) is converted into a binary label, where 0 represents fresh fruit and 1 represents rotten fruit.

Finally, the pixel values are scaled. The images contain pixel values from 0 to 255, whereas MobileNetV2 was pre-trained with inputs in the range −1 to 1. A rescaling layer is therefore placed inside the model, which computes x / 127.5 − 1. This process is equivalent to the standard MobileNetV2 preprocessing function. Because the rescaling is part of the model, the same preprocessing is automatically applied during training, testing and prediction in the web application.

IMAGE AUGMENTATION

Data augmentation creates slightly modified versions of the training images during training, which exposes the model to more variety and helps reduce overfitting. The following random transformations are applied:

- Horizontal flip
- Rotation of up to about ±10% of a full turn (36 degrees)
- Zoom of up to 15%
- Contrast change of up to 10%

Augmentation is applied to the training set only. The validation and test images are not modified, so that they measure performance on real, unaltered images. Illustrative example: a photograph of a banana may be flipped and slightly zoomed in one epoch and rotated in the next, so the network sees different versions of the same fruit while the label stays the same.

DATASET SPLITTING

The notebook divides the prepared dataset into three subsets:

- Training set: 70% of the data, used to learn the model parameters.
- Validation set: 15% of the data, used to monitor performance during training and to choose the best model.
- Testing set: 15% of the data, used for the final evaluation on unseen images.

The first split reserves 30% of the data for validation and testing. This portion is then divided equally into two subsets. Stratified splitting by fruit type and freshness is used so that every subset contains similar proportions of each type. The split is performed on the list of image files before any augmentation, and the notebook checks that no image appears in more than one subset. This prevents the same image file, or an augmented copy of it, from appearing in more than one subset. Because the split is made by image file and not by individual fruit, near-

duplicate photographs of the same fruit (if the dataset contains any) could still fall into different subsets; this is discussed under Limitations.

The notebook uses a fixed random seed (42) to improve reproducibility. With all 3,200 images readable, the 70/15/15 split gives 2,240 training, 480 validation and 480 test images. The numbers in the table below follow from the split settings, assuming 200 images in each of the 16 fruit-and-freshness groups; the actual counts are printed when the notebook is executed and should be checked against that output.

Subset	Images	Fresh	Rotten
Training set (70%)	2,240	1,120	1,120
Validation set (15%)	480	240	240
Test set (15%)	480	240	240

CNN Model Architecture (MobileNetV2)

The proposed model uses the following architecture:

Input Layer
Image of size 224 × 224 × 3 (RGB, pixel values 0–255)

Rescaling Layer
Scales pixel values to the range −1 to 1 (x / 127.5 − 1)

MobileNetV2 Base
Pre-trained on ImageNet, without its original classification layer; outputs 7 × 7 × 1280 feature maps

GlobalAveragePooling2D
Converts the feature maps into a vector of 1280 values

Dropout Layer
Dropout rate: 0.3

Output Layer
One neuron with sigmoid activation: probability that the fruit is rotten

EXPLANATION OF THE LAYERS

1. **Input layer:** Accepts the fruit image as an array of 224 × 224 pixels with three colour channels.
2. **Rescaling layer:** Converts pixel values from the range 0–255 to the range −1 to 1, as required by MobileNetV2. Keeping this step inside the model ensures that prediction uses the same preprocessing as training.
3. **MobileNetV2 base:** A lightweight convolutional network of about 2.26 million parameters, pre-trained on ImageNet. It uses depthwise separable convolutions and

inverted residual blocks with linear bottlenecks to extract features such as edges, textures and shapes efficiently.

4. **Global average pooling layer:** Averages each of the 1280 feature maps into a single value, producing a compact vector and reducing the number of parameters compared with flattening.
5. **Dropout layer:** Randomly switches off 30% of the values during training to reduce reliance on particular features and help reduce overfitting.
6. **Output layer:** Contains one neuron with sigmoid activation. Its output is a value between 0 and 1 that the system interprets as the estimated probability that the fruit is rotten. A value of 0.5 or higher gives the class Rotten; otherwise the class is Fresh.

MODEL TRAINING

The model is trained using the training dataset, while the validation dataset is used to monitor learning performance. The notebook uses the Adam optimizer to update the model parameters and binary cross-entropy as the loss function, which is suitable for a classification task with two classes. Class weights are calculated from the training data so that both classes contribute fairly to the loss.

Training is performed in two phases:

1. **Phase 1 (head training):** The MobileNetV2 base is frozen, so its pre-trained weights are not changed. Only the new classification head is trained, using a learning rate of 1e-3 for a maximum of 10 epochs.
2. **Phase 2 (fine-tuning):** The last 30 layers of the MobileNetV2 base are unfrozen and trained together with the head, using a very small learning rate of 1e-5 for a maximum of 8 epochs. The batch normalization layers remain frozen to keep training stable.

The batch size is 32. An epoch represents one complete pass through the training dataset, while the batch size determines how many images are processed together before a parameter update.

Two callbacks are used in each phase. Early stopping monitors the validation loss and stops training when it does not improve for three consecutive epochs, restoring the best weights. Model checkpointing saves the model with the lowest validation loss. After both phases, the notebook compares the two saved models using validation data and selects the better one. The test set is never used for this choice.

These settings help manage training time and reduce the risk of overfitting. The final number of epochs actually completed depends on the training behaviour.

IMPLEMENTATION DETAILS

The project is implemented in Python using TensorFlow/Keras for deep learning, Scikit-learn for dataset splitting and evaluation, and Streamlit and Gradio for the user interface. The implementation is divided into the following modules:

1. **Library installation and imports:** Loads the required Python packages.
2. **Dataset loading:** Downloads the raw subset of the fresh and rotten fruit dataset from the Hugging Face Hub.

3. **Data preparation:** Exports resized images into fresh and rotten folders and creates the binary labels.
4. **Exploratory data analysis:** Displays class counts per fruit type and sample images.
5. **Data splitting:** Creates stratified training, validation and test sets and checks that they do not overlap.
6. **Input pipeline:** Builds tf.data pipelines with augmentation applied to the training data only.
7. **Model construction:** Builds the MobileNetV2-based classification model.
8. **Model training:** Trains the head, then fine-tunes the top layers, with early stopping and checkpointing.
9. **Performance evaluation:** Calculates classification metrics and plots the training curves, confusion matrix and ROC curve.
10. **Model saving:** Stores the trained model (.keras file) and the class names.
11. **Web application:** A Gradio interface in the notebook and a Streamlit application (app.py) that accept an image and return the predicted class and confidence, with clear messages for invalid files. Both interfaces show a low-confidence warning when the confidence is below 70%, and the Streamlit application also accepts a photograph taken with the device camera.
12. **Deployment:** The Streamlit application (app.py) and the requirements file (streamlit, tensorflow-cpu, numpy and Pillow) are stored in a GitHub repository. The trained model file (food_freshness_model.keras), produced by the notebook, is placed in the model/ folder so that the application can load it. The repository is prepared for deployment on Streamlit Community Cloud.

ALGORITHM

Input: A labelled dataset of fresh and rotten fruit images.

Output: A trained model that predicts whether a fruit image shows fresh or rotten fruit.

Algorithm steps:

1. Start the program.
2. Import the required libraries.
3. Download the raw subset of the Fresh and Rotten Fruits dataset from the Hugging Face Hub.
4. Dataset: https://huggingface.co/datasets/Project-AgML/fresh_rotten_fruit_classification (original: https://data.mendeley.com/datasets/bdd69gyhv8)
5. Export the images into fresh and rotten folders, resizing each to 224 × 224 pixels.
6. Assign class 0 to fresh fruit and class 1 to rotten fruit.
7. Divide the image files into training, validation and test sets using stratified splitting.

8. Create input pipelines; apply augmentation to the training set only.
9. Build the model: rescaling layer, MobileNetV2 base (frozen), global average pooling, dropout and a sigmoid output layer.
10. Compile the model using the Adam optimizer and binary cross-entropy loss.
11. Phase 1: train the classification head for up to 10 epochs with early stopping and checkpointing.
12. Phase 2: unfreeze the last 30 layers of the base and fine-tune for up to 8 epochs with a learning rate of 1e-5.
13. Select the better of the two saved models using validation loss.
14. Predict class probabilities for the test dataset.
15. Convert probabilities to classes using a threshold of 0.5.
16. Calculate accuracy, precision, recall, F1-score and ROC-AUC.
17. Display the confusion matrix, ROC curve and training curves.
18. Save the trained model.
19. Accept a new fruit image through the web application and predict its class and confidence.
20. Stop the program.

PERFORMANCE EVALUATION METRICS

The project uses several evaluation metrics to measure the quality of fresh/rotten classification. Rotten fruit is treated as the positive class.

1. **Accuracy:** Measures the proportion of all images that are correctly classified.
2. **Precision:** Measures how many fruits predicted as rotten are actually rotten.
3. **Recall:** Measures how many actual rotten fruits are correctly identified.
4. **F1-score:** Combines precision and recall into a single metric using their harmonic mean.
5. **ROC-AUC:** Measures how well the model distinguishes the two classes across different classification thresholds. A higher value generally indicates better class separation.
6. **Confusion matrix:** Shows correct and incorrect predictions for both fresh and rotten fruit.

Here, TP (true positives) means rotten fruit correctly detected, TN (true negatives) means fresh fruit correctly classified, FP (false positives) means fresh fruit incorrectly flagged as rotten, and FN (false negatives) means rotten fruit incorrectly classified as fresh.

RESULTS AND DISCUSSION

The project evaluates the trained MobileNetV2 model using a separate test dataset that was not used for training or for model selection. The model generates the probability that each test

image shows rotten fruit, and a threshold of 0.5 is used to convert the probabilities into binary predictions.

The evaluation module calculates accuracy, precision, recall, F1-score and ROC-AUC. It also generates a classification report, confusion matrix, ROC curve, training and validation curves, and the accuracy for each fruit type. The training curves help to understand how the model learns and whether overfitting occurs.

The confusion matrix provides additional insight into the types of classification errors. False positives represent fresh fruit incorrectly identified as rotten, while false negatives represent rotten fruit incorrectly classified as fresh. For food safety, false negatives are usually the more serious error.

Verification status: The notebook stored in the project repository (Food_Freshness_Detection.ipynb) contains no saved execution output, and the trained model file and metrics.json are not included in the repository. The metric values, confusion-matrix counts, per-fruit accuracies and Figures 2 to 4 reported below are therefore carried over from the earlier draft of this report and have not been independently reproduced. They must be confirmed against the notebook output (or metrics.json) before submission and corrected if they differ.

MODEL EVALUATION RESULTS

Metric	Value
Accuracy	0.9938 (99.38 %)
Precision (Rotten)	0.9958 (99.58 %)
Recall (Rotten)	0.9917 (99.17 %)
F1-score (Rotten)	0.9937
ROC-AUC	0.9997
True negatives (Fresh correctly classified)	239
False positives (Fresh incorrectly flagged)	1
False negatives (Rotten incorrectly missed)	2
True positives (Rotten correctly detected)	238

On the 480 unseen test images the reported accuracy is 99.38 %, with precision 0.9958, recall 0.9917, F1-score 0.9937 and ROC-AUC 0.9997 for the Rotten class. The confusion matrix (Figure 3) shows 1 fresh fruit incorrectly flagged as rotten and 2 rotten fruit incorrectly classified as fresh.

Only 3 of the 480 test images were misclassified: 1 fresh fruit was wrongly flagged as rotten (false positive) and 2 rotten fruits were missed (false negatives). Precision for the Rotten class was 0.9958 and recall was 0.9917, so the two kinds of error were similarly rare. Because missing a rotten fruit is the more serious error for food safety, the 2 false negatives are the errors that matter most in practice.

Figure 2 shows the full 10 head-training epochs followed by 8 fine-tuning epochs (18 epochs in total). The training and validation curves stay close together throughout, which suggests that overfitting was limited. Validation accuracy rose from about 95% at the end of phase 1 to about 98.7% after fine-tuning, which suggests that unfreezing the top 30 layers of MobileNetV2 helped; no separate experiment was run to isolate the effect of fine-tuning.

Per-fruit results are shown in the table below. Apple, banana, orange, pomegranate and strawberry were classified with 100% accuracy, and grape, guava and jujube each had one error (98.3%). Each fruit type has only 60 test images, so a single mistake changes its accuracy by about 1.7 percentage points, and these small differences should not be over-interpreted.

These results are very high, but they come from a single dataset split of one published dataset. Photographs from the same shop or session may share similar backgrounds, and because the split is made by image file, near-duplicate photographs of the same fruit (if present) could appear in both the training and test sets. Performance on completely new real-world photographs may therefore be lower. The ROC-AUC of 0.9997 indicates that the model separates the two classes very well across thresholds on this test set.

ACCURACY FOR EACH FRUIT TYPE (TEST SET)

Fruit type	Test images	Accuracy
apple	60	100.0 %
banana	60	100.0 %
orange	60	100.0 %
grape	60	98.3 %
guava	60	98.3 %
jujube	60	98.3 %
pomegranate	60	100.0 %
strawberry	60	100.0 %

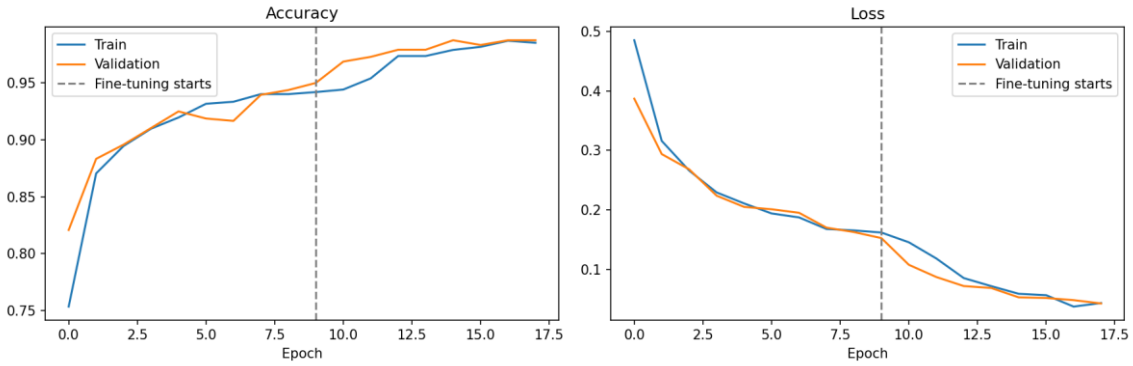


Figure 2: Training and validation accuracy and loss

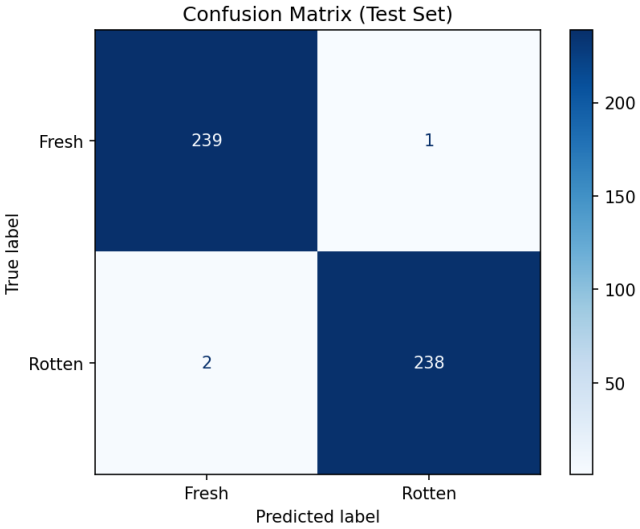


Figure 3: Confusion matrix on the test set

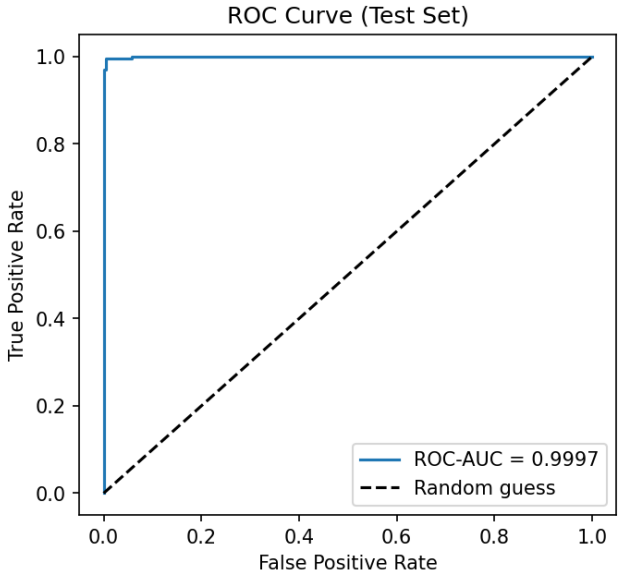


Figure 4: ROC curve on the test set

SCREENSHOT PENDING

Figure 5(a): Streamlit application showing a Fresh prediction, with the uploaded or captured image, the predicted class, the confidence score and the class-probability bars.

SCREENSHOT PENDING
Figure 5(b): Streamlit application showing a Rotten prediction, with the uploaded or captured image, the predicted class, the confidence score and the class-probability bars.

Figure 5: Screenshots of the Streamlit application showing a Fresh prediction (a) and a Rotten prediction (b). Screenshots pending: to be captured by running the application.

APPLICATIONS

1. **Households:** Can help consumers check whether fruit is still good to eat.
2. **Retail and supermarkets:** Can support quick screening of fruit so that rotten items are removed from shelves.
3. **Food waste reduction:** Can help identify spoiled items early, before they affect other produce.
4. **Agriculture and supply chain:** Can serve as a component of grading and sorting systems for harvested fruit.
5. **Smart kitchens and refrigerators:** Can be integrated into devices that monitor the condition of stored food.
6. **Education and research:** Can be used to study transfer learning and binary image classification.

These are potential applications. The project implements model-based image prediction through a web application; it does not itself provide a conveyor-belt sorting system or a smart refrigerator.

ADVANTAGES

- Learns visual features directly from fruit images.
- Uses transfer learning, which reduces the training data and time required.
- MobileNetV2 is lightweight and fast.
- Applies data augmentation, dropout and early stopping to help control overfitting.
- Uses validation data to monitor training and select the model, and a separate test set for final evaluation.
- Evaluates performance using multiple complementary metrics.
- Provides an easy-to-use web interface with a confidence score and error handling.
- Allows the trained model to be saved, reloaded and deployed online.

LIMITATIONS

- The model can produce false positive and false negative predictions.

- The model recognizes only the eight fruit types in the dataset; other fruits or non-fruit images may produce confident but incorrect results.
- The dataset is relatively small (3,200 original images), so the test set contains only a few hundred images and the measured scores carry some uncertainty.
- Photographs from the same shop or session may share similar backgrounds, so results on completely new real-world images may be lower than the test results.
- Because the dataset was split by image file and not by individual fruit, near-duplicate photographs of the same fruit, if present, may appear in both the training and test sets, which could make the test scores higher than the performance on genuinely new fruit.
- The evaluation is based on a single dataset split; repeated runs or cross-validation were not performed, and no other architecture was trained for comparison.
- The task is binary; the system does not estimate ripeness stages or the remaining shelf life.
- The predicted probability is a model estimate and is not necessarily a calibrated measure of real-world risk.
- Performance depends on the quality, lighting and focus of the photograph.
- The system should not be used as the sole basis for deciding whether food is safe to eat.

FUTURE ENHANCEMENTS

The project can be extended in several ways:

1. **Dataset expansion:** Train and evaluate the model on more fruits, vegetables and real-world images with varied backgrounds and lighting.
2. **Ripeness and shelf-life estimation:** Extend the classes to unripe, ripe, overripe and rotten, or predict remaining shelf life.
3. **Model comparison:** Compare MobileNetV2 with other models such as VGG16, ResNet50, EfficientNet and a CNN trained from scratch.
4. **Explainability:** Add Grad-CAM heat-maps to show which regions of the fruit influenced the prediction.
5. **Out-of-distribution detection:** Reject images that do not contain a supported fruit.
6. **Mobile deployment:** Convert the model to TensorFlow Lite for a mobile application or an embedded device.
7. **Real-time detection:** Use object detection to find and classify several fruits in one image or video stream.
8. **Improved evaluation:** Test on externally sourced data to study generalization to new conditions.
9. **Threshold tuning:** Select the classification threshold based on the relative costs of false positives and false negatives.
10. **Integration:** Integrate the model with smart storage, sorting or inventory systems.

CONCLUSION

This project presents a food freshness detection system that classifies fruit images as fresh or rotten using a MobileNetV2-based convolutional neural network. The system uses the Fresh and Rotten Fruits dataset, performs preprocessing and augmentation, and trains the model using transfer learning in two phases.

The model architecture includes a rescaling layer, a pre-trained MobileNetV2 base, global average pooling, dropout and a sigmoid output layer. The model is trained using the Adam optimizer and binary cross-entropy loss, with early stopping and checkpointing to manage training. The data is split before augmentation and the augmented copies of the dataset are not used, which avoids leakage through augmented copies of the same photograph.

The implementation evaluates the trained model using accuracy, precision, recall, F1-score, ROC-AUC and a confusion matrix. It also provides a Streamlit web application, prepared for deployment through GitHub, that predicts the class of an uploaded fruit image and shows a confidence score. The reported results on the 480 unseen test images are 99.38% accuracy and a ROC-AUC of 0.9997, with only 3 misclassified images; these values are subject to the verification status stated in the Results and Discussion section.

The project demonstrates how deep learning and transfer learning can be applied to food quality assessment. Although the system can assist in identifying potentially rotten fruit, its predictions are not definitive safety verdicts. Further testing on diverse and more recent datasets would be necessary before practical use.

REFERENCES

1. N. Sultana, M. Jahan and M. S. Uddin, “An extensive dataset for successful recognition of fresh and rotten fruits,” Data in Brief, vol. 44, Art. no. 108552, 2022, doi: 10.1016/j.dib.2022.108552. Dataset: “Fresh and Rotten Fruits Dataset for Machine-Based Evaluation of Fruit Quality,” Mendeley Data, V1, doi: 10.17632/bdd69gyhv8.1. Available at: https://data.mendeley.com/datasets/bdd69gyhv8
2. Project-AgML, fresh_rotten_fruit_classification (Hugging Face dataset, CC BY 4.0). Available at: https://huggingface.co/datasets/Project-AgML/fresh_rotten_fruit_classification
3. M. Sandler, A. Howard, M. Zhu, A. Zhmoginov and L.-C. Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2018, pp. 4510–4520.
4. J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, “ImageNet: A large-scale hierarchical image database,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2009, pp. 248–255.
5. TensorFlow, TensorFlow Documentation. Available at: https://www.tensorflow.org/
6. Keras, Keras API Documentation. Available at: https://keras.io/
7. Scikit-learn, Machine Learning in Python. Available at: https://scikit-learn.org/
8. Streamlit Documentation. Available at: https://docs.streamlit.io/

9. A. Abid et al., “Gradio: Hassle-free sharing and testing of ML models in the wild,” arXiv:1906.02569, 2019.
10. Python Software Foundation, Python Documentation. Available at: https://docs.python.org/3/